

Section A: Conceptual Application

1. Flow of Request in Spring MVC (JSP → Controller → DB → View)

Spring MVC follows a structured flow to process user requests.

Step-by-Step Flow:

1. User submits form (JSP Page)

- User enters data in a JSP form and clicks submit.
- The request is sent to the server.

2. Request reaches DispatcherServlet

- It acts as the **front controller**.
- It receives all incoming requests.

3. Handler Mapping

- DispatcherServlet checks which controller should handle the request.
- It maps URL to the appropriate controller method.

4. Controller Processing

- The controller method processes the request.
- It retrieves form data using model attributes.

5. Business Logic / Service Layer

- Controller calls service or DAO layer.

6. Database Interaction (DAO Layer)

- DAO uses JdbcTemplate or JPA to interact with database.
- Data is fetched or stored.

7. Return to Controller

- DAO returns result to controller.

8. Model and View Creation

- Controller adds data to Model object.
- Returns logical view name (e.g., "dashboard").

9. View Resolver

- Resolves logical name to actual JSP file
- Example: "dashboard" → /WEB-INF/views/dashboard.jsp

10. Response to User

- Final JSP page is rendered and sent to browser.

2. IoC (Inversion of Control) and Dependency Injection

IoC (Inversion of Control):

- IoC means **control of object creation is given to Spring container instead of programmer.**
- Instead of creating objects manually using new, Spring creates and manages them.

👉 Example:

```
UserService service = new UserService();
```

In Spring:

- Spring container creates and manages UserService.

Dependency Injection (DI):

- DI is a technique used to achieve IoC.
- It means **injecting required objects (dependencies) into a class.**

👉 Example:

```
class A
```

```
{  
    B b;  
}
```

Instead of creating B inside A, Spring injects B into A.

Types of DI:

1. Constructor Injection

2. Setter Injection

Advantages:

- Loose coupling
- Easy testing
- Better code maintainability

3. Difference between BeanFactory and ApplicationContext

Feature	BeanFactory	ApplicationContext
Type	Basic container	Advanced container
Bean Loading	Lazy loading	Eager loading
Features	Limited	Full features (AOP, events, i18n)
Performance	Faster (lightweight)	Slightly slower
Usage	Rarely used	Widely used

Explanation:

- **BeanFactory** is the simplest container in Spring.
- **ApplicationContext** extends BeanFactory and provides more features.

👉 Example:

```
ApplicationContext context = new ClassPathXmlApplicationContext("config.xml");
```

4. JdbcTemplate in Spring

JdbcTemplate is a class in Spring that **simplifies JDBC operations**.

Problems with Traditional JDBC:

- Need to load driver

- Open/close connection manually
- Handle exceptions
- Write repetitive code

How JdbcTemplate helps:

- Automatically manages connection
- Handles exceptions
- Reduces boilerplate code

Example:

```
String sql = "SELECT * FROM users WHERE username=? AND password=?";  
List<User> users = jdbcTemplate.query(sql, new Object[]{u, p}, rowMapper);
```

Advantages:

- Less code
- Better readability
- Automatic resource management

5. Role of DispatcherServlet in Spring MVC

DispatcherServlet is the **core component (Front Controller)** in Spring MVC.

Main Responsibilities:

1. Receives all incoming requests
2. Finds appropriate controller using HandlerMapping
3. Calls controller method
4. Sends response to ViewResolver
5. Returns final view to user

Flow:

Client → DispatcherServlet → Controller → ViewResolver → JSP → Response

Importance:

- Centralized request handling
- Simplifies application structure
- Improves maintainability

6. Constructor Injection vs Setter Injection

Constructor Injection:

- Dependencies are injected using constructor.

```
class A
{
    B b;
    public A(B b) {
        this.b = b;
    }
}
```

Setter Injection:

- Dependencies are injected using setter methods.

```
class A
{
    B b;
    public void setB(B b) {
        this.b = b;
    }
}
```

Difference Table:

Feature	Constructor Injection	Setter Injection
Injection Time	At object creation	After object creation
Mandatory Dependency	Yes	No
Immutability	Yes	No
Flexibility	Less flexible	More flexible

Conclusion:

- Use **Constructor Injection** for required dependencies
- Use **Setter Injection** for optional dependencies